1. What is React? (Basic Theory)

- Definition: React is a JavaScript Library (not a framework) used to build User Interfaces (UI).
- Main Use: It is used to create Single Page Applications (SPA) where the page does not reload when data changes.
- Component-Based: The website is broken down into small parts called Components (like Header, Sidebar, Button). These components are put together to make the full website (like LEGO blocks).
- Virtual DOM: React creates a copy of the actual DOM (website structure). When data changes, it only updates the specific part that changed, not the whole page. This makes it very fast.
- JSX: It stands for JavaScript XML. It allows us to write HTML inside JavaScript.

---

2. History & Developer

- Created By: Jordan Walke (A Software Engineer at Facebook).
- Inspiration: It was influenced by "XHP" (a PHP framework).
    - 2011: First used in Facebook's Newsfeed.
    - 2012: Used in Instagram.
    - May 2013: Released to the public (Made Open Source)

3. Advantages (Pros)

---

1. High Performance: Because of the Virtual DOM, the app runs faster.
2. Reusable Components: You can write a component (like a button) once and use it many times across the app. This saves time.
3. One-Way Data Flow: Data flows only in one direction (Parent to Child). This makes it easier to find and fix errors (debugging).
4. Strong Community: It is very popular, so there are many tutorials and solutions available online.
5. React Native: If you learn React, you can also build mobile apps (Android/iOS) using React Native.

---

4. Disadvantages (Cons)

1. Fast Paced: React updates very quickly. Developers have to keep learning new things constantly.
2. JSX can be confusing: For beginners, mixing HTML and JavaScript in one file (JSX) can look difficult at first.
3. "View" Only: React only handles the UI (what you see). For other things like Routing or storing data (State Management), you need to learn extra libraries like Redux or React Router.
4. Poor Documentation: Because it changes so fast, sometimes the official guides feel outdated.

5. Interesting Facts
   1. Original Name: Before it was named React, Jordan Walke called it "FaxJS".
   2. The Instagram Moment: Facebook decided to make React open-source because they wanted Instagram to use it too.
   3. Big Companies Use It: It is not just Facebook; Netflix, Airbnb, Uber, and WhatsApp web all use React.
   4. Most Loved: It is consistently voted as one of the most loved and used web technologies in developer surveys.

React JS – Installation & Requirements

A. Basic Requirements (Prerequisites)

Before you install React, you must have these things ready on your computer.

1. Knowledge Requirements (What you should know)
   - HTML & CSS: You need to know how to structure and style a webpage.
   - JavaScript (ES6): React is built on JavaScript. You must know basics like:
     - Variables (let, const)
     - Arrow Functions (() => {})
     - Array Methods (.map(), .filter())
     - Destructuring.

2. Software Requirements (What to download)

- Node.js:
  - *Explanation:* React needs a special environment to run outside the browser during development. Node.js provides this.
  - *Action:* Download "LTS (long term support)" version from nodejs.org.
- NPM (Node Package Manager):
  - *Explanation:* Think of this as an "App Store" for coding. It helps you download React and other tools. It installs automatically when you install Node.js.
- Code Editor:
  - *Recommendation:* VS Code (Visual Studio Code). It is the best editor for React.

B. Installation Process (Line-by-Line Explanation)

Today, the modern and fastest way to install React is using a tool called Vite (pronounced "Veet"). The old way (create-react-app) is now slow and not recommended.

Step 1: Open Terminal
Open your Command Prompt (CMD), PowerShell, or the VS Code Terminal.

Step 2: Check Node Version
Type this command and press Enter:

```
node -v
```

- Explanation: This checks if Node.js is installed.
- Result: It should show a version number (e.g., v18.16.0). If it shows an error, you haven't installed Node.js.

Step 3: Create the Project
Type this command to create a new React app:

```
npm create vite@latest
```

- npm: Calls the Node Package Manager tool.
- create: Tells npm to start a new project.
- vite@latest: Uses the latest version of the Vite tool (which is faster than the old React tools).

(After pressing Enter, it will ask you a few questions):

1. Project Name: Type a name (e.g., my-first-app).
2. Select Framework: Choose React (use arrow keys).
3. Select Variant: Choose JavaScript (or JavaScript + SWC(speedy web compiler)).

Step 4: Enter the Project Folder
Type this command:

```
cd my-first-app
```

- cd: Stands for Change Directory.
- my-first-app: The name of the folder you just created.
- Meaning: You are telling the terminal to "go inside" your project folder.

Step 5: Install Dependencies (The libraries)
Type this command:

```
npm install
```

- Explanation: When you created the folder, it was empty to save space. This command goes to the internet and downloads all the necessary files (React library, dom tools, etc.) into a folder called node_modules.
- Note: This might take 1 or 2 minutes.

Step 6: Start the Server
Type this command:

```
npm run dev
```

- npm run: Tells npm to execute a script.
- dev: Starts the "Development Server".
- Result: You will see a link like http://localhost:5173.

Step 7: View in Browser

- Action: Control+Click the link (http://localhost:5173) or open Chrome and type it in.
- Result: You will see the React logo spinning. Congratulations! Your React App is running.

C. Important Files (What did you just install?)

When you open the folder in VS Code, you will see these important files:

1. node_modules/: A huge folder containing all the downloaded code libraries. Never edit this.
2. public/: Contains static files like images or the favicon (logo).
3. src/ (Source): This is where you work.
    - App.jsx: The main component of your website.
    - main.jsx: The entry point that connects React to the HTML.
4. package.json: The "ID Card" of your project. It lists the project name, version, and all the libraries you are using.

Use Arrow Up and Arrow Down to select a turn, Enter to jump to it, and Escape to return to the chat.

(For Online Editor ViteJS.dev)

React File Structure – Explained

When you open your project in VS Code, it looks something like this:

```
my-react-app/
├── node_modules/       (Folder)
```

```
├── public/          (Folder)
├── src/             (Folder)
├── index.html       (File)
├── package.json     (File)
├── package-lock.json (File)
└── vite.config.js   (File)
```

Here is the explanation of each part:

1. The Folders

- *node_modules/*
  - What is it? This is the Warehouse.
  - Explanation: When you ran npm install, it downloaded React, Vite, and thousands of other small JavaScript tools that your project needs to run.
  - Important: Never touch this folder. It is very heavy (large size). If you delete it, the app breaks. If you want to share your code, never send this folder (the other person can re-download it using package.json).
- *public/*
  - What is it? The Public Storage.
  - Explanation: Files put here are served exactly as they are.
  - Use: You keep static assets like images, logos (favicon), or fonts here that don't need to change.
- *src/ (Source)*
  - What is it? The Kitchen / Workspace.
  - Explanation: 99% of your work happens here. This is where you write your React components, CSS styles, and logic.

2. The Root Files (Configuration)

index.html

- What is it? The Frame of your website.
- Explanation: This is the ONLY HTML file in the entire project.
- Key Line: Inside the <body>, there is a .
- How it works: React takes your entire application and "injects" it inside this empty div. You rarely edit this file.

package.json

- What is it? The ID Card / Receipt.
- Explanation:
  - It tells the Project Name and Version.
  - It lists "Dependencies": The list of libraries your project needs (like React, React-DOM).
  - It lists "Scripts": The commands to run the app (like npm run dev).

package-lock.json

- What is it? The Detailed Receipt.
- Explanation: While package.json says "We need React", this file says "We need exactly React version 18.2.0 downloaded from this specific link." It ensures that everyone working on the project has the *exact* same version of tools.

vite.config.js

- What is it? The Settings.
- Explanation: This contains configuration for the Vite tool (like which port to run on, or how to handle plugins). Beginners usually don't need to touch this.

3. Inside the src Folder (Your Code)

main.jsx (or index.js)

- What is it? The Entry Point / Switch.
- Explanation: This is the first JavaScript file that runs.
- Job: It finds the in your HTML file and puts the <App /> component inside it. It connects React to the Browser.

App.jsx

- What is it? The Main Container.
- Explanation: This is your root component. All other components (like Navbar, Footer, Login Form) are put inside this file.
- Visual: Think of App.jsx as the "Home Page" structure.

App.css / index.css

- What is it? The Styling.
- Explanation:
  - index.css: Usually holds global styles (like background color for the whole body).
  - App.css: Usually holds styles specific to the App component.

Summary Flow: How they connect

1. Browser opens index.html.
2. index.html calls main.jsx.
3. main.jsx grabs App.jsx.
4. App.jsx displays your design on the screen.

Here is the simplest "Hello World" program for React.

We will modify the main file in your project to show a simple message.

Step 1: Open the File

Go to your project folder: src > App.jsx.

(Delete everything inside this file so it is completely empty).

Step 2: Write this Code

Copy and paste this code into App.jsx:

```
function App() {
  return (
    <div># <> or <Fragment> also use Fragment Tag which is more better
      <h1>Hello, World!</h1>
      <p>This is my first React program.</p>
    </div># </> or </Fragment> closing Fragment
  );
}
export default App;
```

Step 3: Run the Project

1. Open your terminal.
2. Type: npm run dev
3. Open the link (e.g., http://localhost:5173) in your browser.

Output: You will see "Hello, World!" and the paragraph text on your white screen.

Line-by-Line Explanation

1. function App() { ... }

- In React, a component is just a simple JavaScript Function.
- The function name (App) must start with a Capital Letter.

2. return (...);
   - This determines what will show up on the screen.
   - Inside the return, we write JSX (which looks like HTML).
3. ...
   - Rule: React components can only return one parent element.
   - We wrap the <h1> and <p> inside a single to follow this rule.
4. export default App;
   - This allows the file main.jsx to import this code and send it to the browser.

Here is a simple example showing multiple components inside one file.

In this example, we have 3 components: Navbar, Footer, and App. However, we only export App as the default.

The Code (src/App.jsx)

```
function Navbar() {
  return (
    <nav style={{ background: "lightblue", padding: "10px" }}>
      <h2>Website Menu</h2>
    </nav>
  );
}
function Footer() {
  return (
    <footer style={{ background: "lightgray", padding: "10px" }}>
```

```
      <p>Copyright 2024</p>
    </footer>
  );
}
function App() {
  return (
    <div>
      <Navbar />
      <div style={{ padding: "20px" }}>
        <h1>Welcome to React</h1>
        <p>We are using multiple components in one file.</p>
      </div>
      <Footer />
    </div>
  );
}
export default App;
```

Explanation

1. Helper Components (Navbar, Footer):
    - We created two small functions.
    - These are NOT exported. They are "private" to this file. They are just used to help the main component.
2. Main Component (App):
    - This component acts like a Container.
    - It uses the helper components by writing them like HTML tags: <Navbar /> and <Footer />.
3. export default App;
    - This tells React: *"When other files try to import this file, give them the App component."*

- The Navbar and Footer are hidden inside; they only appear because App is using them.

Key Rule to Remember

A file can have many Components (functions), but it can have only one export default.
What is a Named Export?
Unlike "Default Export" (which only allows one main item per file), Named Export allows you to export multiple things (components, variables, or functions) from a single file.

The Code Example
Imagine we have a file called Tools.jsx where we keep small reusable components.

File 1: src/Tools.jsx (Sending)
We put the word export in front of every function we want to share.

```
export function MyButton() {
  return <button>Click Me</button>;
}
export function MyHeader() {
  return <h1>Top Headline</h1>;
}
export const appName = "My Cool App";
```

File 2: src/App.jsx (Receiving)
To use these, we must use Curly Braces { } and the Exact Names

```
// We import specific items using { }
import { MyButton, MyHeader, appName } from './Tools';
```

```
function App() {
  return (
    <div>
      <MyHeader />
      <p>Welcome to {appName}</p>
      <MyButton />
    </div>
  );
}
export default App;
```

Key Rules of Named Exports

1. Multiple Exports: You can export 10, 20, or 100 items from one file.
2. Curly Braces { }: When importing, you MUST use curly braces.
    - *Correct:* import { MyButton } from ...
    - *Wrong:* import MyButton from ... (This is for default exports only).
3. Exact Names: The name must match exactly.
    - If you export function MyButton, you cannot import it as import { SimpleBtn }. It has to be { MyButton }.

Difference: Default vs Named

Feature	Default Export	Named Export
Quantity	Only 1 per file	Many per file
Import Syntax	import AnyName from ...	import { RealName } from ...
Braces	No braces	{ Uses Braces }
Naming	Can rename it freely	Must use exact name

Here is a simple example of how to export a value (like a String, Number, or Array) instead of a component.

This is very useful for storing Constants (data that doesn't change, like a website name, color code, or API link).

1. The File with Values (src/Data.js)

We use export const to send the values out.

codeJavaScript

```
export const websiteName = "React Masterclass";
export const version = 5.0;
export const courses = ["HTML", "CSS", "JavaScript", "React"];
```

2. The File using the Values (src/App.jsx)

We import them using Curly Braces { } because they are named exports.

```
import { websiteName, version, courses } from './Data';

function App() {

  return (
    <div>
      <h1>Welcome to {websiteName}</h1>
      <p>Current Version: {version}</p>
      <h3>Courses Available:</h3>
      <ul>
        {/* displaying the first item in the array */}
        <li>{courses[0]}</li>
        <li>{courses[1]}</li>
      </ul>
    </div>
  );
}
export default App;
```

What is JSX?

- Full Form: JavaScript XML.
- Definition: It is a syntax extension for JavaScript. It allows us to write HTML-like code inside a JavaScript file.
- Why use it? It makes writing the User Interface (UI) very easy and readable.

Note: Browsers (Chrome, Firefox) do not understand JSX. Tools like Vite (SWC) or Babel convert JSX into regular JavaScript that the browser can read.

1. Basic Syntax (HTML inside JS)

```
function App() {
  // This looks like HTML, but it is actually JavaScript code!
  const message = <h1>Hello, this is JSX!</h1>;
  return message;
}
export default App;
```

2. Embedding Variables (The Power of JSX)

You can put dynamic JavaScript variables inside HTML using Curly Braces { }.

```
function App() {
  const name = "Ajay";
  const age = 25;
  return (
    <div>
      <h1>Welcome, {name}</h1>
      <p>Your age is {age}</p>
      <p>In 5 years, you will be {age + 5}.</p>
```

```
    </div>
  );
}
export default App;
```

3. The 3 Rules of JSX

Rule A: The Single Parent Rule

You cannot return multiple elements side-by-side. You must wrap them in one parent tag (like a or a Fragment <>).

 Wrong:

```
return (
  <h1>Hello</h1>
  <p>World</p>
);
// Error: JSX expressions must have one parent element.
```

Correct:

```
return (
  <div>
    <h1>Hello</h1>
    <p>World</p>
  </div>
);
```

Rule B: camelCase Attributes

Since we are writing inside JavaScript, some HTML words are reserved.

- You cannot use class. You must use className.
- You cannot use for. You must use htmlFor.
- Event handlers use camelCase: onclick becomes onClick.

```
<div className="container">
  <button onClick={handleClick}>Click Me</button>
</div>
```

Rule C: Closing Tags

In HTML, some tags like <img> or <input> don't need closing. In JSX, every tag must be closed.

```
// □ HTML Way (Errorin React)
<img src="logo.png">
<input type="text">
// JSX Way (Self-closing)
<img src="logo.png" />
<input type="text" />
```

4. JSX vs Regular JavaScript (Under the Hood)

What you write (JSX):

```
const element = <h1 className="title">Hello</h1>;
```

What React actually sees (Regular JS):

```
const element = React.createElement(
  'h1',
  { className: 'title' },
  'Hello'
);
```

Conclusion: JSX allows us to write the first version (easy), and the compiler turns it into the second version (complex) for the browser.

In React (JSX), there are two different ways to call a function, depending on when you want it to run.

1. Call Immediately (To Display Data)
If you want the function to run automatically when the page loads, you call it with parentheses ().
Syntax: { myFunction() }

```
function App() {
   function getGreeting() {
    return "Good Morning!";
  }
  return (
    <div>
   <h1> {getGreeting()} </h1>
    </div>
  );
}
export default App;
```

2. Call on Event (To Respond to User)

If you want the function to run only when the user clicks (or types), you pass the function name without parentheses.

Syntax: onClick={ myFunction }

```
function App() {
    function showMessage() {
    alert("You clicked the button!");
  }
  return (
    <div>
      <button onClick={showMessage}>Click Me</button>
    </div>
  );
}
export default App;
```

3. How to Pass Arguments (Values) to a Function?

Syntax: onClick={ () => myFunction("Data") }

```
function App() {
  function sayHello(name) {
    alert("Hello " + name);
  }
  return (
    <div>
     <button onClick={() => sayHello("Ajay")}>Say Hello to Ajay</button>
     <button onClick={() => sayHello("Rahul")}>Hello to Rahul</button>
```

```
    </div>
  );
}
export default App;
```

Summary Table

Goal	Syntax	Example
Show result on screen	With ()	{getName()}
Run on Click (No Data)	No ()	<button onClick={run}>
Run on Click (With Data)	Arrow Function	<button onClick={() => run(5)}>

1. User Variable with JSX

You can display the value of any JavaScript variable inside your HTML.

Example:

```
function App() {
  const name = "Rahul";
  const age = 25;

  return (
    <div>
      {/* React replaces {name} with "Rahul" */}
      <h1>Name: {name}</h1>
```

```
      <p>Age: {age}</p>
    </div>
  );
}
```

2. Condition inside JSX

You cannot use if-else statements inside curly braces (because they don't return a value). Instead, we use the Ternary Operator (? :) or Logical AND (&&).

```
function App() {
  const isLoggedIn = true;
  return (
    <div>
      {/* Syntax: condition ? true_result : false_result */}
      <h1>
        {isLoggedIn ? "Welcome Back!" : "Please Login"}
      </h1>

      {/* Logical AND: Shows text only if condition is true */}
      {isLoggedIn && <button>Logout</button>}
    </div>
  );
}
```

3. Use Function with JSX

You can call a function inside curly braces. React will run the function and display whatever it returns.

Example:

```
function App() {
  function getGreeting(user) {
    return "Hello " + user;
  }
  return (
    <div>
      {/* Calling the function immediately */}
      <h2>{getGreeting("Ajay")}</h2>
    </div>
  );
}
```

4. Operations inside JSX

You can perform mathematical calculations or string combinations directly inside the braces.

```
function App() {
  const a = 10;
  const b = 20;
  return (
    <div>
      <p>The sum is: {a + b}</p>      {/* Output: 30 */}
      <p>Multiplication: {a * 10}</p> {/* Output: 100 */}
      <p>Random: {Math.random()}</p>  {/* Calls Math Library */}
    </div>
  );
```

```
}
```

5. Object and Array with JSX

A. Arrays: React can display arrays automatically.

B. Objects: Important Rule! You cannot display a full object directly. You must display specific properties (like .name).

Example:

```
function App() {
  const fruits = ["Apple", "Mango", "Banana"];
  const person = { name: "Riya", city: "Delhi" };

  return (
    <div>
          <p>{fruits}</p> {/* Output: AppleMangoBanana */}
      <p>User Name: {person.name}</p>
    </div>
  );
}
```

6. HTML Tags Property with JSX

You can use variables to control HTML attributes

(like src, href, placeholder, style, disabled). When using a variable, do not use quotes "".

Example:

```
function App() {
  const imgLink = "https://via.placeholder.com/150";
  const isButtonDisabled = true;

  return (
    <div>
      {/* Passing variable to src attribute */}
      <img src={imgLink} alt="Demo" />

      {/* Passing boolean to disabled attribute */}
      <button disabled={isButtonDisabled}>Click Me</button>
    </div>
  );
}
```

7. Interview Question

Question: Can we use a for loop or an if-else statement inside JSX curly braces { }?

Answer:
No, we cannot.

JSX curly braces can only handle Expressions (code that produces a value immediately).

- if-else and for loops are Statements (they control logic but don't evaluate to a value directly).
- Solution: To loop, we use .map(). To use conditions, we use ternary operators ? :.

Wrong:

```
return <div> { if(true) { return "Hi" } } </div> // □ Error
```

Correct:

```
return <div> { true ? "Hi" : "Bye" } </div> // □ Works
```

State in React JS

In standard JavaScript, if you change a variable, the HTML (UI) does not update automatically.

The Problem (Without State):

```
function App() {
  let count = 0; // Normal variable
  function increase() {
    count = count + 1;
    console.log(count); // The console shows 1, 2, 3... BUT the screen stays 0
  }
  return (
    <div>
      <h1>{count}</h1>
      <button onClick={increase}>Click Me</button>
    </div>
  );
}
```

Why? React doesn't know that count changed, so it doesn't refresh (re-render) the screen. State is required to tell React: "Data has changed, please update the screen."

2. What is State?

- Definition: State is a built-in React object that is used to contain data or information about the component.
- Simple Logic: Think of state as a component's Memory.
- Behavior: Whenever the state changes, the component Re-renders (refreshes) automatically to show the new data.

3. What are Hooks?

In modern React (Functional Components), we cannot use this.state (which was used in old Class components).

Instead, we use special functions called Hooks.

- Hooks allow us to "hook into" React features like state and lifecycle methods.
- All hooks start with the word use.
- The hook to manage state is called useState.

4. How to use State (Syntax)

To use state, you must import useState from React.

```
const [variableName, setFunctionName] = useState(initialValue);
```

1. variableName: Holds the current data (e.g., count).
2. setFunctionName: A function used to update the data (e.g., setCount).
3. initialValue: The starting value (e.g., 0).

5. Example of State (Practical)

Here is the corrected counter example that actually updates the screen.

```
import { useState } from 'react'; // 1. Import Hook
function App() {
  const [data, setData] = useState(0);

  function updateData() {
    setData(data + 1);
```

```
  }
  return (
    <div>
      <h1>Count: {data}</h1>
      <button onClick={updateData}>Update Data</button>
    </div>
  );
}
export default App;
```

Result: When you click, setData runs -> React notices the change -> React refreshes the screen -> data becomes 1.

6. State in Different Components

State is Local and Private to the component. If you reuse the same component multiple times, each one has its own independent state. Changing one does not affect the other.

Practical Example:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
```

```
}
function App() {
  return (
    <div>
      <h3>Component 1:</h3>
      <Counter />
      <h3>Component 2:</h3>
      <Counter />
    </div>
  );
}
export default App;
```

Observation: If you click the first button, only the first number changes. The second button stays at 0.

7. Multiple States

You are not limited to one state variable. You can create as many states as you need in a single component.

Practical Example:

```
function App() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState("Rahul");
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increase Count</button>
```

```
      <h1>Name: {name}</h1>
      <button onClick={() => setName("Ajay")}>Change Name</button>
    </div>
  );
}
```

8. Interview Question

Q1: Why can't we update the state directly like data = data + 1?

Answer:
If you modify the variable directly (e.g., data = 5), React does not know that the data has changed. Therefore, it will not trigger a re-render, and the UI will not update. You must use the setter function (e.g., setData(5)) to trigger the update cycle.

Q2: What is the argument passed to useState()?

Answer:
The argument passed to useState(ARGUMENT) is the initial state (the starting value) of the variable. It is used only during the very first render.

Step 1, 2, & 3: The Complete Code Example
Let's combine "Define State", "Update State", and "Add Condition" into one simple example.

```
import { useState } from 'react';

function App() {
  const [show, setShow] = useState(true);

  return (
    <div style={{ margin: "20px" }}>
          {
       show ? <h1>Hello World!</h1> : null
      }
     <button onClick={() => setShow(!show)}>
       Toggle / Hide-Show
     </button>
          <button onClick={() => setShow(!show)}>
       {show ? "Hide Text" : "Show Text"}
     </button>
    </div>
  );
}
export default App;
```

Example:

```
function UserProfile() {
  return (
    <div style={{ background: "lightblue", padding: "10px" }}>
```

```
      <h2>User: Rangeesh</h2>
      <p>Email: rangeesh@example.com</p>
    </div>
  );
}
function App() {
  const [status, setStatus] = useState(false); // Initially hidden
  return (
    <div>
      <button onClick={() => setStatus(!status)}>
        {status ? "Hide Profile" : "Show Profile"}
      </button>
      {status && <UserProfile />}
    </div>
  );
}
export default App;
```

Practice Question :Try to create a "Read More / Read Less" feature.

1. Create a long paragraph of text.
2. Create a state called isExpanded.
3. If isExpanded is true, show the full paragraph.
4. If isExpanded is false, show only the first line.
5. Add a button that changes text between "Read More" and "Read Less". in react

```
import { useState } from "react";
```

```jsx
function ReadMore() {
  const [isExpanded, setIsExpanded] = useState(false);

  const text =
    "React is a popular JavaScript library used for building user interfaces. " +
    "It allows developers to create reusable UI components. " +
    "React uses a virtual DOM which improves performance. " +
    "It is maintained by Meta and a large developer community.";

  return (
    <div style={{ width: "400px", margin: "20px auto", fontFamily: "Arial" }}>
      <p>
        {isExpanded ? text : text.substring(0, 80) + "..."}
      </p>

      <button onClick={() => setIsExpanded(!isExpanded)}>
        {isExpanded ? "Read Less" : "Read More"}
      </button>
    </div>
  );
}

export default ReadMore;
```

: Second Method :

```
import React, { useState } from "react";
function ReadMoreIfExample() {
  const [isExpanded, setIsExpanded] = useState(false);

  const paragraph =
    "React is a JavaScript library used to build fast and interactive user interfaces. " +
    "It follows a component-based architecture which makes code reusable and easy to maintain. " +
    "React uses a virtual DOM to improve performance and efficiency. " +
    "It is widely used in modern web applications.";

  let displayText;
  let buttonText;

  // IF CONDITION LOGIC
  if (isExpanded === true) {
    displayText = paragraph;          // full text
    buttonText = "Read Less";
  } else {
    displayText = paragraph.substring(0, 90) + "..."; // short text
    buttonText = "Read More";
  }

  return (
    <div style={styles.container}>
      <h2>Read More / Read Less Example</h2>
      <p>{displayText}</p>
      <button onClick={() => setIsExpanded(!isExpanded)}>
        {buttonText}
```

```
      </button>
    </div>
  );
}

export default ReadMoreIfExample;

// Inline CSS
const styles = {
  container: {
    width: "450px",
    margin: "40px auto",
    fontFamily: "Arial",
    lineHeight: "1.6",
    border: "1px solid #ccc",
    padding: "20px",
    borderRadius: "6px"
  }
};
```

Program 1: Login Status (if–else)

```
import React, { useState } from "react";
function LoginStatus() {
  const [isLoggedIn, setIsLoggedIn] = useState(false);
  let message;
  let buttonText;

  if (isLoggedIn === true) {
    message = "Welcome User";
    buttonText = "Logout";
  } else {
    message = "Please Login";
    buttonText = "Login";
  }

  return (
    <div>
      <h2>{message}</h2>
      <button onClick={() => setIsLoggedIn(!isLoggedIn)}>
        {buttonText}
      </button>
    </div>
  );
}
export default LoginStatus;
```

Program 2: Pass / Fail Result

```
import React from "react";

function ResultCheck() {
  const marks = 55;
  let result;

  if (marks >= 40) {
    result = "Pass";
  } else {
    result = "Fail";
  }

  return (
    <div>
      <h3>Marks: {marks}</h3>
      <h2>Result: {result}</h2>
    </div>
  );
}

export default ResultCheck;
```

Program 3: Age Validation (Multiple if)

```
import React from "react";

function AgeCheck() {
  const age = 20;
  let message;

  if (age < 18) {
    message = "Access Denied";
  } else if (age >= 18 && age < 60) {
    message = "Access Granted";
  } else {
    message = "Senior Citizen Access";
  }

  return (
    <div>
      <h2>Age: {age}</h2>
      <h3>{message}</h3>
    </div>
  );
}

export default AgeCheck;
```

Program 4: Loading / Error / Success

```
import React from "react";

function ApiStatus() {
  const isLoading = false;
  const isError = false;
  let output;

  if (isLoading === true) {
    output = "Loading...";
  } else if (isError === true) {
    output = "Error Occurred!";
  } else {
    output = "Data Loaded Successfully";
  }

  return (
    <div>
      <h2>{output}</h2>
    </div>
  );
}

export default ApiStatus;
```

Program 5: Button Text & Color Change

```
import React, { useState } from "react";

function ToggleButton() {
  const [isOn, setIsOn] = useState(false);
  let text;
  let bgColor;

  if (isOn === true) {
    text = "ON";
    bgColor = "green";
  } else {
    text = "OFF";
    bgColor = "red";
  }

  return (
    <button
      style={{ backgroundColor: bgColor, color: "white", padding: "10px" }}
      onClick={() => setIsOn(!isOn)}
    >
      {text}
    </button>
  );
}

export default ToggleButton;
```

Program 6: Greeting Based on Time

```
import React from "react";

function Greeting() {
  const hour = new Date().getHours();
  let greeting;

  if (hour < 12) {
    greeting = "Good Morning";
  } else if (hour < 17) {
    greeting = "Good Afternoon";
  } else {
    greeting = "Good Evening";
  }

  return <h2>{greeting}</h2>;
}

export default Greeting;
```

Step 6: Interview Questions

Q1: What is the difference between Conditional Rendering (React) vs CSS display: none?

- CSS (display: none): The element is still present in the DOM (HTML code), it is just invisible to the eye. It still takes up memory.
- React Conditional Rendering ({show && <Tag />}): The element is completely removed from the DOM. It does not exist in the HTML until you show it again. This is better for performance in large apps.

Q2: Which operator is best for Toggling?

- If you want to show something OR show nothing, use the Logical AND (&&) operator. (e.g., {show && <Box />}).
- If you want to show something OR show something else (like "Login" vs "Logout" button), use the Ternary Operator (? :). (e.g., {isLoggedIn ? <Logout /> : <Login />}).

If-Else-If

1. const [count, setCount] = useState(0)
    - We create a variable count starting at 0.
2. onClick={() => setCount(count + 1)}
    - Every time you click the "Counter" button, the number increases by 1 (0 -> 1 -> 2...).

2. Practical Code

```
import { useState } from "react";
function App() {
  const [count, setCount] = useState(0);
  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>Current Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Counter ++
      </button>
      {
        count == 0 ? <h1>Condition 0 (Start)</h1>
        : count == 1 ? <h1>Condition 1</h1>
        : count == 2 ? <h1>Condition 2</h1>
        : count == 3 ? <h1>Condition 3</h1>
        : null  // If count is 5 or more, show nothing
      }
    </div>
  );
}
export default App;
```

3. Pro-Tip (Best Practice)

While the code in the image works, Nested Ternary Operators (linking many ? : ? :) are considered bad practice in real jobs because they are hard to read.

If you have more than 2 conditions, it is better to use a standard if/else above the return statement.

```
function App() {
  const [count, setCount] = useState(0);
  // Calculate the message before returning HTML
  let message = null;
  if (count === 0) message = <h1>Start</h1>;
  else if (count === 1) message = <h1>One</h1>;
  else if (count === 2) message = <h1>Two</h1>;
  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Count</button>
      {message}  {/* Simply display the result */}
    </div>
  )
}
```

Props(Properties)

- Definition: Props are used to pass data from one component (Parent) to another component (Child).
- Analogy: Think of a Component like a Function and Props like Arguments (Parameters).
- Rule: Props are Read-Only (Immutable). The child component receives them but cannot change them.

Program 1: Passing Simple Props (Name & Age)

```
import React from "react";
import Student from "./Student";
function App() {
  return (
    <div>
      <Student name="Rahul" age={20} />
      <Student name="Anita" age={22} />
    </div>
  );
}
export default App;
import React from "react";                    Child Component
function Student(props) {
  return (
    <div>
      <h3>Name: {props.name}</h3>
      <p>Age: {props.age}</p>
    </div>
  );
}
export default Student;
```

Program 2: Props with Destructuring

```
import React from "react";
<Employee empName="Ajay" salary={45000} />
```

```
function Employee({ empName, salary }) {
  return (
    <div>
      <h3>Employee Name: {empName}</h3>
      <p>Salary: ₹{salary}</p>
    </div>
  );
}
export default Employee;
```

Program 3: Passing Function as Props

Parent Component

```
import React from "react";
import Child from "./Child";

function Parent() {
  const showMessage = () => {
    alert("Hello from Parent Component");
  };

  return <Child clickHandler={showMessage} />;
}

export default Parent;
```

Child Component

```
import React from "react";

function Child(props) {
  return (
    <button onClick={props.clickHandler}>
      Click Me
    </button>
  );
}
export default Child;
```

Program 4: Props with if Condition (Age Check)

```
import React from "react";

function AgeStatus({ age }) {
  let message;

  if (age >= 18) {
    message = "Adult";
  } else {
    message = "Minor";
  }

  return (
    <h2>Age Status: {message}</h2>
  );
}
```

```
export default AgeStatus;
Usage:

<AgeStatus age={16} />
```

Program 5: Reusable Button Using Props

```
import React from "react";
function CustomButton({ text, color }) {
  return (
    <button
      style={{
        backgroundColor: color,
        color: "white",
        padding: "10px",
        margin: "5px"
      }}
    >
      {text}
    </button>
  );
}

export default CustomButton;
Usage:
<CustomButton text="Save" color="green" />
<CustomButton text="Delete" color="red" />
```

Program 6: Passing Array as Props

```
import React from "react";
function StudentList({ students }) {
  return (
    <ul>
      {students.map((name, index) => (
        <li key={index}>{name}</li>
      ))}
    </ul>
  );
}
export default StudentList;
Usage:

<StudentList students={["Aman", "Riya", "Kunal"]} />
```

Program 7: Using props.children

This example demonstrates a very special and powerful feature in React called "Children Props" (or creating a Wrapper Component).

```
import React from "react";

function Card(props) {
  return (
    <div style={{ border: "1px solid gray", padding: "15px" }}>
      {props.children}
    </div>
  );
}

export default Card;
```

```
Usage:

<Card>
  <h2>Hello Students</h2>
  <p>This is React Props Example</p>
</Card>
```

Program 8. Passing Objects and Arrays

```
Parent (App.jsx):
function App() {
   const address = "Mumbai, India";
  const details = { age: 25, mobile: "9999999999" };
  return (
    <div>
  <Profile address={address} otherDetails={details} />
    </div>
  );
}
Child (Profile.jsx):
function Profile(props) {
  return (
    <div>
          <h3>Address: {props.address}</h3>
          <p>Age: {props.otherDetails.age}</p>
          <p>Mobile: {props.otherDetails.mobile}</p>
    </div>
  );
}
```

Program 9. Pass Data with Click (Update Props)

Props themselves cannot be changed by the Child. But if the Parent changes its State, the new State value is passed down as Props, and the Child updates automatically.

Example: changing a Name on Button Click

```
import { useState } from 'react';
// Child Component
function UserCard(props) {
  return (
    <div style={{ border: "1px solid black", padding: "10px" }}>
      <h2>User Name: {props.name}</h2>
    </div>
  )
}
// Parent Component
function App() {
  const [name, setName] = useState("Vijay"); // Initial State
  return (
    <div style={{ padding: 20 }}>
      <h1>Props with State</h1>
      {/* Pass the State "name" as a prop */}
      <UserCard name={name} />
      {/* Button updates State -> Which automatically updates Prop */}
      <button onClick={() => setName("Soni")}>Change Name</button>
      <button onClick={() => setName("Vijay")}>Reset</button>
    </div>
  );
}
export default App;
```

 Important Points for Students

✔ Props are read-only
✔ Used for data transfer (Parent → Child)
✔ Makes components reusable
✔ Cannot be modified inside child

5. Interview Questions
Q1: What is the difference between State and Props?

- State: It is internal data. It is managed *inside* the component. It can be changed (using useState).
- Props: It is external data. It is passed *from parent to child*. It is Read-Only (cannot be changed by the child).

Q2: Can a Child component modify its own Props?

- Answer: No. Props are immutable. If a child tries to do props.name = "New Name", it will throw an error. Only the Parent can change the data being sent.

Q3: Can we pass a Function as a Prop?

- Answer: Yes! This is very common. The parent passes a function to the child, and the child calls that function (usually on a button click) to send data back up to the parent.

Handling Input Fields

This code is a perfect practical example of "Handling Input Fields" in React using State. It shows how to capture what a user types and display it live on the screen.

```
import { useState } from 'react';
function App() {
    const [val, setval] = useState("");
            const handleChange = (event) => {
            setval(event.target.value);
            };
    const clearData = () => {
            setval("");
            };
  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <label>Enter Value: </label>
      <input
        type="text"
        placeholder="Enter Name..."
        value={val}
        onChange={handleChange}
        style={{ padding: "10px", fontSize: "16px", margin: "10px" }}/>
      <br />
      <button onClick={clearData}
        style={{
          color: "red",
```

```
            }}
      >Click Me (Clear)
      </button>
      <div style={{ marginTop: "20px", color: "blue" }}>
        <h1>Output: {val}</h1>
      </div>
    </div>
  );
}
export default App;
```

Program 1: Live Input Display

```
import { useState } from "react";
function LiveInput() {
  const [text, setText] = useState("");

  return (
    <div>
      <input onChange={(e) => setText(e.target.value)} />
      <p>You typed: {text}</p>
    </div>
  );
}

export default LiveInput;
```

Program 2: Input Validation

```
function Validation() {
  const [text, setText] = useState("");
  let message = text.length < 5 ? "Minimum 5 characters" : "Valid";
  return (
    <>
      <input onChange={(e) => setText(e.target.value)} />
      <p>{message}</p>
    </>
  );
}
```

Program 3: Form Submit Without Reload

```
function Form() {
  const [name, setName] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
    alert(name);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input onChange={(e) => setName(e.target.value)} />
      <button>Submit</button>
    </form>
  );
}
```

Checkbox

A checkbox is created like this:

<input type="checkbox" />

---

2. Create State for Checkbox

We use state to control the checkbox value (true/false).

const [agree, setAgree] = useState(false);

- false → not checked
- true → checked

3. Get Checkbox Value in State

We use onChange to update the state when the user clicks the checkbox.

```
<input
  type="checkbox"
  checked={agree}
  onChange={(e) => setAgree(e.target.checked)}
/>
```

Full Simple Example

```
import { useState } from "react";
```

```
function App() {
  const [agree, setAgree] = useState(false);
  return (
    <div>
      <h2>Handle Checkbox in React</h2>
          <input type="checkbox" checked={agree}
                  onChange={(e) =>setAgree(e.target.checked)}/>
        I agree to terms & conditions
      </label>

      <h3>Checkbox Value: {agree ? "Checked" : "Not Checked"}</h3>
    </div>
  );
}

export default App;
```

4. Remove Checkbox Value (Multiple Checkboxes)

When you have many checkboxes, you store selected items in an array.

Example: Select Skills

```
import { useState } from "react";
function Skills() {
  const [skills, setSkills] = useState([]);
  const handleCheckbox = (e) => {
    const value = e.target.value;
    if (e.target.checked)
```

```
      setSkills([...skills, value]);   // Add skill
    else
      setSkills(skills.filter((skill) => skill !== value));
  };
  return (
  <div>
  <h2>Select Skills</h2>
  <input type="checkbox" value="HTML" onChange={handleCheckbox} /> HTML
  <br />
  <input type="checkbox" value="CSS" onChange={handleCheckbox} /> CSS
  <br />
  <input type="checkbox" value="JS" onChange={handleCheckbox} />JS
  <h3>Selected Skills: {skills.join(", ")}</h3>
  </div>
  );
}
export default Skills;
```

Program 1: Single Checkbox (Show / Hide Text)

```
import React, { useState } from "react";
function ShowHideCheckbox() {
  const [isChecked, setIsChecked] = useState(false);
  let message;
  if (isChecked === true) {
    message = "Checkbox is checked!";
  } else {
    message = "Checkbox is unchecked!";
  }
  return (
```

```
    <div>
    <input type="checkbox" checked={isChecked} onChange={() => setIsChecked(!isChecked)}/>
      <label> Accept Terms</label>
      <h3>{message}</h3>
    </div>
  );
}
export default ShowHideCheckbox;
```

Program 2: Enable / Disable Button Using Checkbox

```
import React, { useState } from "react";
function EnableButton() {
 const [agree, setAgree] = useState(false);

  return (
    <div>
        <input type="checkbox" checked={agree} onChange={() => setAgree(!agree)}/>
        <label> I agree to terms</label>
          <button disabled={!agree}>
        Submit
        </button>
    </div>
  );
}

export default EnableButton;
```

Program 3: Multiple Checkboxes (Select Hobbies)

```
import React, { useState } from "react";
function Hobbies() {
  const [hobbies, setHobbies] = useState([]);
  const handleChange = (e) => {
    const value = e.target.value;
    if (hobbies.includes(value)) {
      setHobbies(hobbies.filter(item => item !== value));
    } else {
      setHobbies([...hobbies, value]);
    }
  };

  return (
    <div>
      <h3>Select Hobbies</h3>
      <input type="checkbox" value="Cricket" onChange={handleChange} /> Cricket <br />
      <input type="checkbox" value="Music" onChange={handleChange} /> Music <br />
      <input type="checkbox" value="Reading" onChange={handleChange} /> Reading <br />
      <h4>Selected Hobbies:</h4>
      <p>{hobbies.join(", ")}</p>
    </div>
  );
}

export default Hobbies;
```

Program 4: Checkbox Validation (Form)

```
import React, { useState } from "react";
function CheckboxValidation() {
  const [agree, setAgree] = useState(false);
  const [error, setError] = useState("");
  const handleSubmit = () => {
    if (agree === false) {
      setError("Please accept terms and conditions");
    } else {
      setError("");
      alert("Form Submitted Successfully");
    }
  };
  return (
    <div>
      <input  type="checkbox"  checked={agree}  onChange={() => setAgree(!agree)}/>
      <label> Accept Terms</label>
      <button onClick={handleSubmit}>
        Submit
      </button>
      <p style={{ color: "red" }}>{error}</p>
    </div>
  );
}

export default CheckboxValidation;
```

Program 5: Checkbox with Props (Reusable Component)

```
import React from "react";
function CustomCheckbox({ label, checked, onChange }) {
```

```
  return (
    <div>
      <input type="checkbox" checked={checked} onChange={onChange} />
      <label> {label}</label>
    </div>
  );
}
export default CustomCheckbox;
<CustomCheckbox label="Subscribe Newsletter" checked={true} onChange={() => alert("Changed")}
/>
 Key Learning Points (For Students)
✔ Checkbox uses checked (not value)
✔ Controlled component using state
✔ Multiple checkbox → array/object
✔ Useful in forms, filters, permissions
```

Radio Buttons

Radio buttons allow the user to select ONLY ONE option.

Example:

```
<input type="radio" name="gender" value="Male" />
<input type="radio" name="gender" value="Female" />
```

Program 1: Basic Radio Button (Select Gender)

```
import React, { useState } from "react";
function GenderSelect() {
  const [gender, setGender] = useState("");
  return (
    <div>
      <h3>Select Gender</h3>
      <input type="radio" name="gender" value="Male" onChange={(e) => setGender(e.target.value)}/> Male
      <input type="radio" name="gender" value="Female" onChange={(e) => setGender(e.target.value)} /> Female
      <input type="radio" name="gender" value="Other" onChange={(e) => setGender(e.target.value)} /> Other
      <h4>Selected Gender: {gender}</h4>
    </div>
  );
}
export default GenderSelect;
```

Program 2: Radio Button with checked (Controlled)

```
import React, { useState } from "react";
function PaymentMode() {
  const [mode, setMode] = useState("Cash");
  return (
    <div>
    <h3>Payment Mode</h3>
<input type="radio" name="pay" value="Cash" checked={mode === "Cash"} onChange={(e) => setMode(e.target.value)}/> Cash
<input type="radio" name="pay" value="Card" checked={mode === "Card"} onChange={(e) => setMode(e.target.value)}/> Card
<input type="radio" name="pay" value="UPI" checked={mode === "UPI"} onChange={(e) => setMode(e.target.value)}/> UPI
      <h4>Selected Mode: {mode}</h4>
    </div>
  );
}
export default PaymentMode;
```

Program 3: Radio Button with if Condition

```
import React, { useState } from "react";
function Subscription() {
  const [plan, setPlan] = useState("");
  let price;
  if (plan === "Basic") {
    price = "₹199 / month";
  } else if (plan === "Standard") {
    price = "₹399 / month";
  } else if (plan === "Premium") {
    price = "₹699 / month";
  } else {
    price = "";
  }
  return (
    <div>
      <h3>Select Subscription Plan</h3>
      <input type="radio" name="plan" value="Basic" onChange={(e) => setPlan(e.target.value)} /> Basic
      <input type="radio” name="plan" value="Standard" onChange={(e) => setPlan(e.target.value)} /> Standard
      <input type="radio" name="plan" value="Premium" onChange={(e) => setPlan(e.target.value)} /> Premium
      <h4>Price: {price}</h4>
    </div>
  );
}
export default Subscription;
```

Program 4: Radio Button in Form with Validation

```
import React, { useState } from "react";
function CourseForm() {
  const [course, setCourse] = useState("");
  const [error, setError] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
    if (course === "") {
      setError("Please select a course");
    } else {
      setError("");
      alert("Selected Course: " + course);
    }
  };
  return (
    <form onSubmit={handleSubmit}>
      <h3>Select Course</h3>
      <input type="radio" name="course" value="C" onChange={(e) => setCourse(e.target.value)} /> C Language
      <input type="radio" name="course" value="Java" onChange={(e) => setCourse(e.target.value)} /> Java
      <input type="radio" name="course" value="React" onChange={(e) => setCourse(e.target.value)}/> React
      <button type="submit">Submit</button>
      <p style={{ color: "red" }}>{error}</p>
    </form>
  );
}
export default CourseForm;
```

Program 5: Radio Button with Props (Reusable Component)

```
import React from "react";
function RadioGroup({ label, options, selected, onChange }) {
  return (
    <div>
      <h4>{label}</h4>
 {options.map((opt) => (
 <div key={opt}>
 <input type="radio" name={label} value={opt} checked={selected === opt} onChange={(e) => onChange(e.target.value)}/>
  {opt}
 </div>
 ))}
 </div>
  );
}
export default RadioGroup;
Usage:
const [city, setCity] = useState("");
<RadioGroup label="City" options={["Delhi", "Mumbai", "Pune"]} selected={city} onChange={setCity} />
```

3. Default Selection in Radio Button

const [gender, setGender] = useState("Male");

And mark radio button as checked:

```
<input type="radio" name="gender" value="Male" checked={gender === "Male"} onChange={(e) => setGender(e.target.value)}
/>
```

3. Make Dropdown

1. Simple Select Dropdown (Controlled Component)

Example: Select Your City

```
import React, { useState } from "react";
function SelectExample() {
  const [city, setCity] = useState("");

  return (
    <div>
      <h3>Select City</h3>
      <select value={city} onChange={(e) => setCity(e.target.value)}>
        <option value="">-- Select City --</option>
        <option value="Delhi">Delhi</option>
        <option value="Mumbai">Mumbai</option>
        <option value="Pune">Pune</option>
      </select>
      <p>Selected City: {city}</p>
    </div>
  );
}

export default SelectExample;
```

2.Select with Button Submit

```
import React, { useState } from "react";
function SelectSubmit() {
  const [course, setCourse] = useState("");
  const [result, setResult] = useState("");
  const handleSubmit = () => {
    setResult(course);
  };
  return (
    <div>
      <h3>Select Course</h3>
      <select onChange={(e) => setCourse(e.target.value)}>
        <option value="">-- Select Course --</option>
        <option value="C">C Language</option>
        <option value="Java">Java</option>
        <option value="React">React</option>
      </select>
      <br /><br />
      <button onClick={handleSubmit}>Submit</button>
      <h4>Selected Course: {result}</h4>
    </div>
  );
}
export default SelectSubmit;
```

3 Select Multiple Values

```
import React, { useState } from "react";
function MultiSelect() {
  const [skills, setSkills] = useState([]);
  const handleChange = (e) => {
    const values = Array.from(e.target.selectedOptions, option => option.value);
    setSkills(values);
  };

  return (
    <div>
      <h3>Select Skills</h3>
      <select multiple onChange={handleChange}>
        <option value="HTML">HTML</option>
        <option value="CSS">CSS</option>
        <option value="JavaScript">JavaScript</option>
        <option value="React">React</option>
      </select>
      <p>Selected Skills: {skills.join(", ")}</p>
    </div>
  );
}
export default MultiSelect;
```

4. Select Using Array (Dynamic Options)

```
import React, { useState } from "react";

function DynamicSelect() {
  const courses = ["C", "C++", "Java", "Python", "React"];
  const [course, setCourse] = useState("");
  return (
    <div>
      <h3>Select Course</h3>
      <select onChange={(e) => setCourse(e.target.value)}>
        <option value="">-- Select --</option>
        {courses.map((c, index) => (
          <option key={index} value={c}>
            {c}
          </option>
        ))}
      </select>

      <h4>Selected: {course}</h4>
    </div>
  );
}

export default DynamicSelect;
```

5. Select with Conditional Output (if condition)

```
import React, { useState } from "react";
function SelectIf() {
  const [payment, setPayment] = useState("");

  return (
    <div>
      <h3>Payment Mode</h3>

      <select onChange={(e) => setPayment(e.target.value)}>
        <option value="">-- Select Mode --</option>
        <option value="Cash">Cash</option>
        <option value="Online">Online</option>
      </select>

      {payment === "Cash" && <p>Pay at counter</p>}
      {payment === "Online" && <p>Pay using UPI / Card</p>}
    </div>
  );
}

export default SelectIf;
```

6. Select in Form with Validation

```
import React, { useState } from "react";
function SelectValidation() {
  const [country, setCountry] = useState("");
  const [error, setError] = useState("");
     const handleSubmit = (e) => {
              e.preventDefault();
              if (country === "") {
              setError("Please select a country");
              } else {
              setError("");
              alert("Country Selected: " + country);
              }};
  return (
    <form onSubmit={handleSubmit}>
      <h3>Select Country</h3>
      <select value={country} onChange={(e) => setCountry(e.target.value)}>
        <option value="">-- Select Country --</option>
        <option value="India">India</option>
        <option value="USA">USA</option>
      </select>
      {error && <p style={{ color: "red" }}>{error}</p>}
      <button type="submit">Submit</button>
    </form>
  );
}
export default SelectValidation;
```

Array

1. What is an Array?

An array is a list of items stored in a single variable.

Example:

```
let numbers = [10, 20, 30, 40];
let names = ["Ajay", "Vijay", "Rohit"];
```

✔ An array can store multiple values
✔ Values can be of any type (number, string, object, etc.)

---

2. Make Array (in React component)

```
function App() {
  const students = ["Rohan", "Sohan", "Mohan"];
  return <div></div>;
}
```

---

3. Make Table in JSX using Map

We use .map() to loop through an array and display items.

✔ Display Array in a Table (Practical Example)

```
function App() {
  const students = [
    { id: 1, name: "Rohan", course: "React" },
    { id: 2, name: "Sohan", course: "Python" },
    { id: 3, name: "Mohan", course: "Java" }
  ];

  return (
    <div>
      <h2>Student Table</h2>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Course</th>
          </tr>
        </thead>

        <tbody>
          {students.map((data) => (
            <tr key={data.id}>
              <td>{data.id}</td>
              <td>{data.name}</td>
              <td>{data.course}</td>
            </tr>
```

```
        ))}
      </tbody>
    </table>
  </div>
 );
}

export default App;
```

✔ How it works?

- .map() goes through every student in the array
- Makes one <tr> for each student
- Shows data in table columns
- key helps React uniquely identify each row

4. Use Map Function for Looping (Simple Example)

✔ Loop through numbers

```
function App() {
 const numbers = [10, 20, 30, 40];

 return (
  <div>
   <h2>Numbers List</h2>

   {numbers.map((num) => (
```

```
        <h3>{num}</h3>
      ))}
    </div>
  );
}
```

✔ This prints:
10
20
30
40

5. Another Practical Example: List Items (<li>)

```
function App() {
  const fruits = ["Apple", "Banana", "Mango"];

  return (
    <ul>
      {fruits.map((f, i) => (
        <li key={i}>{f}</li>
      ))}
    </ul>
  );
}
```

Clock Component (Clock.jsx)

```jsx
import { useState, useEffect } from "react";
function Clock(props) {
  const [time, setTime] = useState(new Date());

useEffect(() => {

    const timer = setInterval(() => {
      setTime(new Date());
    }, 1000);

return () => clearInterval(timer);
  }, []);
  return (
    <h1 style={{ color: props.color, textAlign: "center" }}>
      {time.toLocaleTimeString()}
    </h1>
  );
}
export default Clock;
```

```jsx
import Clock from "./Clock";
function App() {
  return (
    <div>
      <h2>React JS Task for Props</h2>
      <Clock color="red" />
      <Clock color="blue" />
      <Clock color="green" />
    </div>
  );
}
export default App;
```

useEffect

useEffect is a React Hook that allows you to run some code automatically:

✔ When the component loads
✔ When a value changes
✔ Repeatedly (like timers)
✔ Cleanup work (stop timers, remove listeners)

Types of Dependencies

1. No Dependency Array

```
useEffect(() => {
  // call every time
})
```

- What happens: If you do not provide a second argument (no array at all), React runs this effect after every single render.
- When to use: This is rarely used because it can cause performance issues. If you update the state inside this effect, you will create an infinite loop.

2. Empty Dependency Array

```
useEffect(() => {
  // call only once
}, [])
```

- What happens: If you pass an empty array [], React runs this effect only once, immediately after the component mounts (loads for the first time).
- When to use: This is very common. Use this for initial setup, like fetching data from an API, starting a timer, or adding an event listener when the page loads.

---

3. Single Dependency

```
useEffect(() => {
  // call on changing single state
}, [state1])
```

- What happens: React keeps track of state1. The effect runs on the first render (mount), and then runs again any time state1 changes.
- When to use: When you want to perform an action in response to a specific variable changing (e.g., validating a password field whenever the user types a new character).

---

4. Multiple Dependencies (State)

```
useEffect(() => {
  // call on changing both state
}, [state1, state2])
```

- What happens: The effect runs on the first render, and then runs again if either state1 OR state2 changes.
- Correction: The comment in the image says "changing both state," which is slightly misleading. It doesn't require *both* to change simultaneously. If just state1 changes, it runs. If just state2 changes, it runs.

- When to use: When your logic depends on a combination of variables (e.g., a search filter that runs if the user changes the "Search Text" OR the "Category" dropdown).

---

5. Multiple Dependencies (Props)

```
useEffect(() => {
  // call on changes props
}, [prop1, props2])
```

- What happens: This works exactly the same as the state example above. useEffect does not care if the variable comes from useState or passed down as props.
- When to use: When a parent component passes down new data, and the child component needs to react to that update.

Dependency Array	Behavior
(Missing)	Runs on every render.
[]	Runs once (on mount).
[data]	Runs when data changes.
[data1, data2]	Runs when data1 OR data2 changes.

Easy Real-Life Example

Example 1: useEffect Running Once (on Component Load)

```
import { useEffect } from "react";

function App() {
  useEffect(() => {
    console.log("Component Loaded!");
  }, []);

  return <h1>Hello</h1>;
}
```

✔ Runs only once
✔ Best for API calls, timers

Example 2: useEffect when State Changes

```
import { useState, useEffect } from "react";

function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Count updated:", count);
  }, [count]);
```

```
  return (
    <>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count + 1)}>Increase</button>
    </>
  );
}
```

✔ Runs every time count changes

Example 3: Fetch API Data on Load

```
useEffect(() => {
  fetch("https://jsonplaceholder.typicode.com/users")
    .then(res => res.json())
    .then(data => console.log(data));
}, []);
```

✔ Runs once
✔ Loads data from server

Same Component, 5 Styling Methods

1. Inline Style

You write style inside the tag using JavaScript object.

```
function InlineExample() {
  return (
    <h2 style={{ color: "blue", backgroundColor: "lightgray", padding: "10px" }}>
      Welcome to React Styling
    </h2>
  );
}
```

 Use When:
✔ Quick styling
  Not good for big components

  Whatis Inline CSS?

Inline CSS means writing CSS styles directly inside the HTML/JSX element.
You don’t use any external CSS file.

 Styling is written inside the style attribute.
 Used for small, quick, or one-time styling.

 Inline CSS in React– Example

```
function App() {
  return (
    <div>
      <h1 style={{ color: "blue", fontSize: "30px" }}>
        This is Inline CSS
      </h1>
      <p style={{ backgroundColor: "yellow", padding: "10px" }}>
        Inline CSS is written inside the element.
      </p>
    </div>
  );
}

export default App;
```

 Why use Inline CSS? (Simple reasons)

✔ For small styling
✔ For quick design
✔ For dynamic style (based on state or props)
✔ No need to create separate CSS files

- CSS properties use camelCase
  Example: background-color → backgroundColor
- Values are inside quotes
- React uses double curly braces:
  style={{ ... }}

```
import { useState } from "react";
function App() {
  const [isActive, setIsActive] = useState(false);
  return (
    <div>
      <button
        style={{
          backgroundColor: isActive ? "green" : "red",
          color: "white",
          padding: "10px 20px",
          border: "none",
          borderRadius: "5px"
        }}
        onClick={() => setIsActive(!isActive)}
      >
        {isActive ? "Active" : "Inactive"}
      </button>
    </div>
  );
}
export default App;
```

✔ Explanation

- If isActive is true, button color = green
- If isActive is false, button color = red
- Clicking the button changes the value of isActive, so the style also changes.

 Another Simple Example (Text Color Change)

```
<h2 style={{ color: marks > 40 ? "green" : "red" }}>
  {marks > 40 ? "Pass" : "Fail"}
</h2>
```

✔ If marks > 40 → text is green
✔ If marks <= 40 → text is red

 One More Simple Example (Box Style)

```
<div
  style={{
    width: "200px",
    height: "100px",
    backgroundColor: darkMode ? "black" : "lightgray",
    color: darkMode ? "white" : "black"
  }}
>
  Theme Box
</div>
```

Inline CSS with Hover Effect Using State

```
import { useState } from "react";
function App() {
  const [hover, setHover] = useState(false);
  return (
    <button
      style={{
        backgroundColor: hover ? "blue" : "gray",
        color: "white",
        padding: "10px 20px",
        border: "none",
        borderRadius: "5px",
        cursor: "pointer"
      }}
      onMouseEnter={() => setHover(true)}
      onMouseLeave={() => setHover(false)}
    >
      Hover Me
    </button>
  );
}

export default App;
```

Another Very Simple Example (Change Text Size on Hover)

```
<p
  style={{
    fontSize: isHover ? "24px" : "18px",
    color: "purple",
    cursor: "pointer"
  }}
  onMouseEnter={() => setIsHover(true)}
  onMouseLeave={() => setIsHover(false)}
>
  Hover to increase size
</p>
```

 Summary

✔ Inline CSS cannot use :hover directly
✔ But you can create hover effects using:

- onMouseEnter
- onMouseLeave
- useState

2 External CSS File

Create a CSS file → write styles → import it → use className.

style.css

```
.heading {
  color: red;
  background: yellow;
  padding: 10px;
}
```

ExternalExample.js

```
import "./style.css";

function ExternalExample() {
  return <h2 className="heading">Welcome to React Styling</h2>;
}
```

 Use When:
✔ Basic styling across many components
 Class names can clash if too many developers working

3 CSS Modules (Most used in real projects)

Creates unique class names automatically → no conflicts.

Heading.module.css

```
.title {
  color: green;
  background: #eee;
  padding: 10px;
}
```

CssModuleExample.js

```
import styles from "./Heading.module.css";

function CssModuleExample() {
  return <h2 className={styles.title}>Welcome to React Styling</h2>;
}
```

 Use When:
✔ Large projects
✔ Avoid style mixing between components

4 Styled Components

You create a component with style inside JavaScript.

npm install styled-components

StyledComponentExample.js

```
import styled from "styled-components";

const Title = styled.h2`
  color: purple;
  background: lightpink;
  padding: 10px;
  border-radius: 8px;
`;

function StyledComponentExample() {
  return <Title>Welcome to React Styling</Title>;
}
```

 Use When:
✔ Dynamic theming (dark/light mode)
✔ Custom UI libraries

5 CSS Library (Example: Bootstrap)

Fastest styling using ready-made classes.

npm install bootstrap

 Add link inside index.js

```
import "bootstrap/dist/css/bootstrap.min.css";
```

BootstrapExample.js

```
function BootstrapExample() {
  return (
    <h2 className="text-white bg-primary p-2">
      Welcome to React Styling
    </h2>
  );
}
```

 Use When:
✔ Speed matters (admin panels, dashboards)
  Design becomes less unique

Practical Summary (Real-World Use)

Style Method	When to Use	Difficulty
Inline	**Very small quick fixes**	
External CSS	**Layout + common styling**	
CSS Modules	**Medium/Big apps**	
Styled Components	**Branding/UI heavy apps**	
CSS Library	**Fast development**	

Project Structure

```
react-styling-demo/
├── src/
│   ├── App.js
│   ├── InlineExample.js
│   ├── ExternalExample.js
│   ├── CssModuleExample.js
│   ├── StyledComponentExample.js
│   ├── BootstrapExample.js
│   ├── style.css
│   ├── Title.module.css
│   └── index.js
│
└── package.json
```

 Step 1: Create React App (if not done)

```
npx create-react-app react-styling-demo
cd react-styling-demo
```

 Step 2: Install Required Library

```
npm install styled-components bootstrap
```

 Step 3: Replace Files with This Code

---

 src/App.js

```
import InlineExample from "./InlineExample";
import ExternalExample from "./ExternalExample";
import CssModuleExample from "./CssModuleExample";
import StyledComponentExample from "./StyledComponentExample";
import BootstrapExample from "./BootstrapExample";

function App() {
  return (
    <div style={{ padding: "20px" }}>
      <InlineExample />
      <ExternalExample />
      <CssModuleExample />
      <StyledComponentExample />
      <BootstrapExample />
    </div>
  );
}

export default App;
```

---

src/InlineExample.js

```
function InlineExample() {
```

```
  return (
    <h2 style={{ color: "blue", background: "#ddd", padding: "10px" }}>
      Inline Styling
    </h2>
  );
}

export default InlineExample;
```

---

src/ExternalExample.js

```
import "./style.css";

function ExternalExample() {
  return <h2 className="external">External CSS Styling</h2>;
}

export default ExternalExample;
```

---

src/style.css

```
.external {
  color: red;
  background: yellow;
  padding: 10px;
}
```

---

src/CssModuleExample.js

```
import styles from "./Title.module.css";

function CssModuleExample() {
  return <h2 className={styles.title}>CSS Module Styling</h2>;
}

export default CssModuleExample;
```

---

src/Title.module.css

```
.title {
  color: green;
  background: #eee;
  padding: 10px;
}
```

---

src/StyledComponentExample.js

```
import styled from "styled-components";

const StyleTitle = styled.h2`
  color: purple;
  background: lightpink;
  padding: 10px;
  border-radius: 6px;
`;
```

```
function StyledComponentExample() {
  return <StyleTitle>Styled Components</StyleTitle>;
}

export default StyledComponentExample;
```

---

src/BootstrapExample.js

```
function BootstrapExample() {
  return (
    <h2 className="text-white bg-primary p-2">Bootstrap Styling</h2>
  );
}

export default BootstrapExample;
```

---

src/index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "bootstrap/dist/css/bootstrap.min.css";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

▶ Step 4: Run Project

useRef is a React hook used to keep a value or get an HTML element without re-rendering the page. useRef is used to access DOM elements or store values without re-rendering the component..

What does useRef actually do?

useRef gives you an **object** with one property:

{ current: value }

- React **keeps this object same** between renders
- You can change current anytime
- Changing current **does not refresh the screen**

Why do we use useRef?

We use useRef when:

1. We need to **access an HTML element directly**
   (input, button, div, etc.)
2. We want to **store data temporarily**
   (previous value, timer id, count, etc.)
3. We want to **avoid unnecessary re-render**

```
import { useRef } from "react";
function  App() {
  const inputRef = useRef(null);
  const inputHandler = () => {
    console.log(inputRef);
    inputRef.current.focus();
    inputRef.current.style.color = "red";
    inputRef.current.placeholder = "enter password";
    inputRef.current.value = "123";
  };
const toggleHandler = () => {
    if (inputRef.current.style.display === "none") {
      inputRef.current.style.display = "inline";
    } else {
      inputRef.current.style.display = "none";
    }
  };
  return (
    <div style={{ padding: "20px" }}>
      <h1>useRef Example</h1>
      <button onClick={toggleHandler}>Toggle</button>
     <input ref={inputRef} type="text" placeholder="Enter user name"/>
     <button onClick={inputHandler}>Focus on Input field</button>
    </div>
  );
}
export default App;
```

Complete React Program (Uncontrolled Component Example)

App.jsx

```
import { useRef } from "react";

function App() {
  const handleForm = (event) => {
    event.preventDefault();
    const user = document.querySelector("#user").value;
    const password = document.querySelector("#password").value;
    console.log("QuerySelector Output:", user, password);
  };
  const userRef = useRef(null);
  const passwordRef = useRef(null);
  const handleFormRef = (event) => {
    event.preventDefault();
    const user = userRef.current.value;
    const password = passwordRef.current.value;
    console.log("useRef Output:", user, password);
  };
  return (
    <div style={{ padding: "20px" }}>
      <h1>Uncontrolled Component</h1>
      {/* Form using document.querySelector */}
      <form onSubmit={handleForm}>
        <h3>Using querySelector()</h3>
        <input type="text" id="user" placeholder="enter user name" />
        <br /><br />
```

```
        <input type="password" id="password" placeholder="enter user password" />
        <br /><br />
        <button type="submit">Submit</button>
      </form>

      <hr />

      {/* Form using useRef */}
      <form onSubmit={handleFormRef}>
        <h3>Using useRef()</h3>
        <input type="text" ref={userRef} placeholder="enter user name" />
        <br /><br />

        <input type="password" ref={passwordRef} placeholder="enter user password" />
        <br /><br />
        <button type="submit">Submit with Ref</button>
      </form>
    </div>
  );
}

export default App;
```

✔ Passing function from parent to child as props
✔ Child component calls parent function
✔ Parent gets value (name) from child
✔ Multiple child components with different names

App.jsx (Parent Component)

```
import User from "./User";

function App() {
  // Parent function
  const displayName = (name) => {
    alert(name);
  };

  return (
    <>
      <h1>Call Parent Component Function From Child Component</h1>

      <User displayName={displayName} name="Vijay" />
      <User displayName={displayName} name="sam" />
      <User displayName={displayName} name="peter" />
  3    <User displayName={displayName} name="bruce" />
    </>
  );
}
```

```
export default App;

User.jsx (Child Component)
function User({ displayName, name }) {
  return (
    <div>
      <button onClick={() => displayName(name)}>
        Display Name
      </button>
    </div>
  );
}

export default User;

 How It Works
✔ Parent passes function → displayName
<User displayName={displayName} name="Vijay" />

✔ Child calls the function with its own name
<button onClick={() => displayName(name)}>Display Name</button>

✔ Parent receives the name and shows alert
alert(name)
```

useTransition

useTransition **is a React hook used to make slow updates run in the background so the app stays fast and responsive.**

In Very Simple Words

- Some updates are **important** (like typing in an input)
- Some updates are **heavy/slow** (like loading a big list)
- useTransition tells React:
  “Do this slow work later, don’t block the user.”

Simple Example

Problem:

User types in input → big list filters → typing becomes slow

Solution: useTransition

```
import { useState, useTransition } from "react";
function App() {
  const [pending, startTransition] = useTransition();
  const handleButton = () => {
    startTransition(async () => {
      // waiting 5 seconds to show loader
      await new Promise((res) => setTimeout(res, 5000));
    });
  };
  return (
    <div style={{ padding: "30px" }}>
      <h1>useTransition Hook in React JS 19</h1>
      {pending ? (
        <img
          style={{ width: "100px" }}
          src="https://cdn.dribbble.com/userupload/23237805/file/original-b01f81713802bdc20e9ce2ba275aec43.gif"
          alt="Loading"
        />
      ) : null}
      <button disabled={pending} onClick={handleButton}>
        Click
      </button>
    </div>
  );
}
export default App;
```

useFormStatus() – React Hook

useFormStatus() is a React hook that helps you check the submission status of a form.
It is mainly used with Server Actions inside React <form> tag.

✔ It tells you:

- Whether the form is submitting
- Whether the form has succeeded
- Whether the form has failed

Why do we use useFormStatus?

To control UI during form submission:

- Disable submit button
- Show a loading spinner
- Show “Submitting…” message
- Prevent double submission

```
"use client";
import { useFormStatus } from "react-dom";

export default function Form() {
  async function handleSubmit() {
    // Simulate async work
    await new Promise(r => setTimeout(r, 2000));
```

```
    console.log("Form Submitted");
  }

  return (
    <form action={handleSubmit}>
      <input name="username" placeholder="Enter Name" />
      <SubmitButton />
    </form>
  );
}

function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button disabled={pending}>
      {pending ? "Submitting..." : "Submit"}
      {pending ? <span>  Loading...</span> : "Submit"}
    </button>
  );
}
```

What is Derived State in React?

Derived State means:

A value that is calculated from existing state, NOT stored separately using useState.

You should not store something in state if you can calculate it from other state.

Because storing extra state creates bugs — the values can become out of sync.

You have only one real state:

const [users, setUsers] = useState([]);

Now we want to show:

1. Total number of users
2. Last user added
3. Unique total users (no duplicates)

 Bad Approach
Create 3 extra state variables:

```
const [total, setTotal] = useState(0);
const [last, setLast] = useState("");
const [unique, setUnique] = useState(0);
```

This is wrong because these values are derived from users.

✔ Correct Approach — *Derived State*
Calculate them directly:

```
const total = users.length;
const last = users[users.length - 1];
const unique = [...new Set(users)].length;
```

Full Working Example (Derived State)

```
import { useState } from "react";

function App() {
  const [users, setUsers] = useState([]);
  const [user, setUser] = useState("");

  const handleAddUser = () => {
    setUsers([...users, user]);
  };

  // Derived state values
  const total = users.length;
  const last = users[users.length - 1];
  const unique = [...new Set(users)].length;

  return (
    <div>
      <h2>Total Users: {total}</h2>
```

```
      <h2>Last User: {last}</h2>
      <h2>Unique Users: {unique}</h2>

      <input
        type="text"
        onChange={(e) => setUser(e.target.value)}
        placeholder="Add new user"
      />

      <button onClick={handleAddUser}>Add User</button>

      {users.map((item, index) => (
        <h4 key={index}>{item}</h4>
      ))}
    </div>
  );
}

export default App;
```

What Is “Lifting State Up” in React?

When two or more components need to share the same state, we do *not* keep that state inside children.

Instead, we lift the state up to the common parent component,
and then pass data or function as props to children.

We have three components:

App.jsx → Parent
AddUser.jsx → Child: takes input & updates state
DisplayUser.jsx → Child: shows the state

```
import { useState } from "react";
import AddUser from "./AddUser";
import DisplayUser from "./DisplayUser";

function App() {
  const [user, setUser] = useState("");
  return (
    <div style={{ padding: "20px" }}>
      <h1>Lifting State Up Example</h1>
      <AddUser setUser={setUser} />
      <DisplayUser user={user} />
    </div>
  );
}
export default App;
```

```
function AddUser({ setUser }) {
  return (
    <div>
      <h2>Add User</h2>
      <input type="text" onChange={(event) => setUser(event.target.value)} placeholder="Enter user name"/>
      <hr />
    </div>
    );
}
export default AddUser;
```

```
function DisplayUser({ user }) {
  return (
    <div>
      <h2>Display User</h2>
      <h3>User Name: {user}</h3>
    </div>
  );
}
export default DisplayUser;
```

What is useActionState?

useActionState is a React hook used to update state when a form is submitted.
You do not need onChange or setState again and again.

Before (using useState)

type → onChange → setState → submit

State updates on every key press.

Now (using useActionState)

submit → action function → state update

State updates only when the form is submitted.

Very Simple Example

```
import React, { useActionState } from "react";

function App() {

  function updateName(prevState, formData) {
    return {
```

```
      name: formData.get("name")
    };
  }

  const [state, formAction] = useActionState(updateName, {
    name: ""
  });

  return (
    <div>
      <h2>Name: {state.name}</h2>
      <form action={formAction}>
        <input name="name" placeholder="Enter name" />
        <button>Submit</button>
      </form>
    </div>
  );
}

export default App;
```

II Second Example

```
import React, { useActionState } from "react";

export default function App() {

  // Action function (runs on form submit)
  const handleSubmit = async (previousData, formData) => {
    const name = formData.get("name");
    const password = formData.get("password");

    // Validation
    if (!name || !password) {
      return { error: "All fields are required" };
    }

    if (password.length < 4) {
      return { error: "Password must be at least 4 characters" };
    }

    // Success response
    return {
      name,
      password,
      message: "Form submitted successfully"
    };
  };

  // useActionState hook
  const [data, action, pending] =
```

```
  useActionState(handleSubmit, undefined);

return (
  <div style={{ padding: "20px" }}>
    <h2>useActionState Hook in React JS</h2>

    <form action={action}>
      <input
        defaultValue={data?.name}
        type="text"
        placeholder="enter name"
        name="name"
      />
      <br /><br />

      <input
        type="password"
        placeholder="enter password"
        name="password"
      />
      <br /><br />

      <button disabled={pending}>
        {pending ? "Submitting..." : "Submit data"}
      </button>
    </form>

    <br />
```

```
      {data?.error && (
        <span style={{ color: "red" }}>{data.error}</span>
      )}

      <br />

      {data?.message && (
        <span style={{ color: "green" }}>{data.message}</span>
      )}

      <h3>Name : {data?.name}</h3>
      <h3>Password : {data?.password}</h3>
    </div>
  );
}
```

What is useId?

useId is a React hook used to create unique IDs.

- It is mainly used to connect <label> with <input>
- It helps accessibility
- It avoids duplicate id problems

Why do we need useId?

Wrong (duplicate id problem)

```
<input id="name" />
<input id="name" />
```

Correct (unique id usinguseId)

```
const nameId = useId();
```

Example :

```
const name = useId();
const password = useId();
const terms = useId();
const skills = useId();
```

Each line creates a unique ID

```
<label htmlFor={name}>Enter User Name</label>
<input id={name} />
```

- htmlFor and id are connected
- Clicking label focuses the input

```jsx
import React, { useId } from "react";

function UserForm() {
  const nameId = useId();
  const passwordId = useId();
  const skillsId = useId();
  const termsId = useId();

  return (
    <div style={{ padding: "20px" }}>
      <h2>User Registration Form</h2>

      <form>
        {/* Name */}
        <label htmlFor={nameId}>Enter User Name</label><br />
        <input id={nameId} type="text" placeholder="Enter name"/>
        <br /><br />

        {/* Password */}
        <label htmlFor={passwordId}>Enter Password</label><br />
```

```
        <input id={passwordId} type="password" placeholder="Enter password"/>
        {/* Skills */}
        <label htmlFor={skillsId}>Enter Skills</label><br />
        <input id={skillsId} type="text" placeholder="Enter skills"/>
        {/* Terms */}
        <input id={termsId} type="checkbox" />
        <label htmlFor={termsId}>Accept Terms & Conditions</label>
      </form>
    </div>
  );
}

export default function App() {
  return <UserForm />;
}
```

Second Example :

```
import React, { useId } from "react";

export default function App() {
  return (
    <div>
      <UserForm />
    </div>
  );
}

function UserForm() {
  const user = useId();      // base unique id
  const terms = useId();     // separate id for checkbox

  return (
    <div style={{ padding: "20px" }}>
      <h2>User Form</h2>

      <form>
        {/* User Name */}
        <label htmlFor={`${user}-name`}>Enter User Name</label><br />
        <inputid={`${user}-name`} type="text" placeholder="enter name"/>
        {/* Password */}
        <label htmlFor={`${user}-password`}>Enter User Password</label><br />
        <input id={`${user}-password`} type="password" placeholder="enter password"/>
        {/* Skills */}
        <label htmlFor={`${user}-skills`}>Enter User Skills</label><br />
```

```
      <input id={`${user}-skills`} type="text" placeholder="enter skills"/>
      {/* Terms & Conditions */}
      <input id={terms} type="checkbox"/>
      <label htmlFor={terms}> Accept Terms and Conditions</label>
    </form>
  </div>
 );
}
```

What is Context API?

The Context API in React is a way to share data between components without passing props manually at every level. Normally, props need to be passed down through every intermediate component, which is called "prop drilling". Context helps avoid that.

```
import { useState } from "react";
import College from "./College";
import { SubjectContext } from "./ContextData";
```

- useState is a React hook to manage state in a functional component.
- College is another component where we might want to access the selected subject.
- SubjectContext is the context object created in ContextData.jsx.

2. Creating State

```
const [subject, setSubject] = useState('');
```

- subject is the current selected subject.
- setSubject updates the subject state when the user selects a subject from the dropdown.

3. Using Context Provider

```
<SubjectContext.Provider value={subject}>
   ...
</SubjectContext.Provider>
```

- The Provider is used to wrap the components that need access to this context.
- value={subject} means the subject state is available to all child components inside this provider.

4. Dropdown to Update State

```
<select defaultValue="" onChange={(event) => setSubject(event.target.value)}>
   <option value="">Select Subject</option>
   <option value="Maths">Maths</option>
   <option value="History">History</option>
   <option value="English">English</option>
</select>
```

- This <select> allows the user to choose a subject.
- onChange updates the subject state whenever the user selects a different option.

5. Using Context in Child Component

```
<College />
```

- The College component can now access the subject without receiving it as a prop, using useContext(SubjectContext) inside College.jsx.

Summary of Flow

1. App.jsx keeps the state subject.
2. App.jsx wraps the child components (College) inside SubjectContext.Provider.
3. The value of subject is passed automatically to any child component that uses SubjectContext.
4. The child component accesses it using useContext(SubjectContext).

Advantages of Context API

- Avoids prop drilling.
- Easy to share global state (like themes, user data, or selected options).
- Cleaner and more maintainable code.

Below is a complete working React Context API program based exactly on the idea shown in your screenshot (Simple, clean, and exam/teaching friendly)

Project Structure

```
src/
├── App.jsx
├── ContextData.jsx
├── College.jsx
└── Student.jsx
```

```
├── Subject.jsx
└── main.jsx
```

---

1 ContextData.jsx

Create Context

```
import { createContext } from "react";

export const SubjectContext = createContext();
```

✔ This file creates a Context object
✔ SubjectContext will be used by Provider and Consumers

---

2 App.jsx

Provide Context value

```
import { useState } from "react";
import College from "./College";
import { SubjectContext } from "./ContextData";

export default function App() {
  const [subject, setSubject] = useState("");
```

```
  return (
    <div style={{ backgroundColor: "yellow", padding: 20 }}>
      <h2>Select Subject</h2>

      <SubjectContext.Provider value={subject}>
        <select defaultValue=" "onChange={(event) => setSubject(event.target.value)}>
          <option value="">Select Subject</option>
          <option value="Maths">Maths</option>
          <option value="History">History</option>
          <option value="English">English</option>
        </select>
        <h1>Context API</h1>
        <College />
      </SubjectContext.Provider>
    </div>
  );
}
```

What happens here?

- subject state is created
- SubjectContext.Provider shares subject
- Any child component can access subject without props

3 College.jsx

Intermediate Component (No Props)

```
import Student from "./Student";

export default function College() {
  return (
    <div style={{ border: "2px solid black", padding: 10, marginTop: 10 }}>
      <h2>College Component</h2>
      <Student />
    </div>
  );
}
```

✔ No props passed
✔ Still data will reach Student

---

4 Student.jsx

Another Child Component

```
import Subject from "./Subject";

export default function Student() {
  return (
    <div style={{ border: "2px solid blue", padding: 10, marginTop: 10 }}>
      <h2>Student Component</h2>
      <Subject />
    </div>
```

```
  );
}
```

✔ Still no props
✔ Context continues flowing

---

5 Subject.jsx

Consume Context

```
import { useContext } from "react";
import { SubjectContext } from "./ContextData";

export default function Subject() {
  const subject = useContext(SubjectContext);
  return (
    <div style={{ border: "2px solid green", padding: 10, marginTop: 10 }}>
      <h2>Subject Component</h2>
      <h3>Selected Subject: {subject}  </h3>
    </div>
  );
}
```

Key Line

```
const subject = useContext(SubjectContext);
```

✔ This directly receives the value
✔ No props needed

---

6 main.jsx

Render App

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Data Flow (Easy Words)

App (state)
↓
Context Provider
↓
College
↓
Student

↓
Subject (useContext)

Why Context API?

- Avoid prop drilling
- Clean code
- Best for global data (theme, user, language, subject, etc.)

React Router

React Router is a library used in React to handle navigation and routing in single-page applications (SPA). It allows you to move between pages without reloading the browser.

Why React Router?

- Create multiple pages/views in a React app
- Fast navigation (no page refresh)
- Clean URLs (e.g. /login, /dashboard)
- Works perfectly with SPA

Install React Router

npm install react-router-dom

Basic Example (React Router v6)

App.js

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Home.js

```
function Home() {
  return <h2>Home Page</h2>;
```

```
}
export default Home;
```

About.js

```
function About() {
  return <h2>About Page</h2>;
}
export default About;
```

Contact.js

```
function Contact() {
  return <h2>Contact Page</h2>;
}
export default Contact;
```

Navigation using Link

```
import { Link } from "react-router-dom";

function Navbar() {
  return (
    <div>
      <Link to="/">Home</Link> |
      <Link to="/about">About</Link> |
      <Link to="/contact">Contact</Link>
    </div>
  );
}
```

```
export default Navbar;
```

□ Navigation using useNavigate() (Programmatic)

```
import { useNavigate } from "react-router-dom";

function Login() {
  const navigate = useNavigate();

  return (
    <button onClick={() => navigate("/dashboard")}>
      Login
    </button>
  );
}

export default Login;
```

React Router – Navbar & Header (Complete Example with CSS)

This example shows how to create a Header + Navbar using React Router v6 with proper design.

Project Structure

```
src/
 ├─ App.js
```

```
├── index.js
├── Navbar.js
├── Home.js
├── About.js
├── Contact.js
└── styles.css
```

---

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./styles.css";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

App.js (Router Setup)

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Navbar from "./Navbar";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

function App() {
  return (
    <BrowserRouter>
```

```
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Navbar.js (Header + Navbar)

```
import { NavLink } from "react-router-dom";

function Navbar() {
  return (
    <header className="header">
      <h2 className="logo">MyApp</h2>
      <nav>
        <NavLink to="/" className="nav-link">Home</NavLink>
        <NavLink to="/about" className="nav-link">About</NavLink>
        <NavLink to="/contact" className="nav-link">Contact</NavLink>
      </nav>
    </header>
  );
}
```

```
export default Navbar;
```

Home.js

```
function Home() {
  return <h1 className="page">Home Page</h1>;
}
export default Home;
```

About.js

```
function About() {
  return <h1 className="page">About Page</h1>;
}
export default About;
```

Contact.js

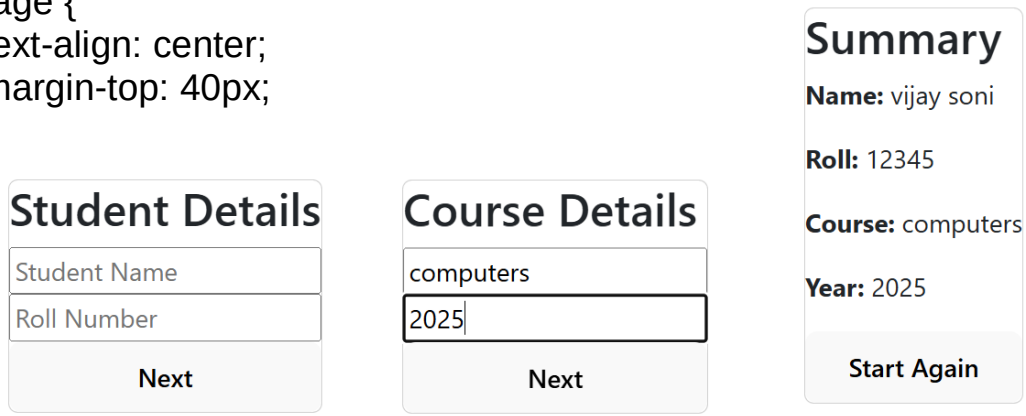
```
function Contact() {
  return <h1 className="page">Contact Page</h1>;
}
export default Contact;
```

styles.css (Navbar + Header Design)

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
}
```

```
.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px 30px;
  background: #1e293b;
  color: white;
}

.logo {
  margin: 0;
}

nav {
  display: flex;
  gap: 20px;
}

.nav-link {
  color: white;
  text-decoration: none;
  font-size: 16px;
}

.nav-link:hover {
  text-decoration: underline;
}
```

```
.page {
  text-align: center;
  margin-top: 40px;
}
```



Step 1: create App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import FormOne from "./FormOne";
import FormTwo from "./FormTwo";
import Summary from "./Summary";

function App() {
return (
<>
<Routes>
<Route path="/" element={<FormOne />} />
<Route path="/form2" element={<FormTwo />} />
<Route path="/summary" element={<Summary />} />
</Routes>
</>
);
```

```
}

export default App;
```

Step 2: Create Formone.jsx

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
function FormOne()
{
const [name, setName] = useState("");
const [roll, setRoll] = useState("");
const navigate = useNavigate();

const nextPage = (e) => {
e.preventDefault();
navigate("/form2", { state: { name, roll } });
};
return (
<div className="container">
<form className="card" onSubmit={nextPage}>
<h2>Student Details</h2>
<input placeholder="Student Name" value={name} onChange={(e) => setName(e.target.value)} />
<input placeholder="Roll Number" value={roll} onChange={(e) => setRoll(e.target.value)} />
<button>Next</button>
</form>
</div>
);
```

```
}
export default FormOne;
```

step 3: Create Formtwo.jsx

```
import { useState } from "react";
import { useLocation, useNavigate } from "react-router-dom";


function FormTwo() {
const location = useLocation();
const navigate = useNavigate();
const [course, setCourse] = useState("");
const [year, setYear] = useState("");


const nextPage = (e) => {
e.preventDefault();
navigate("/summary", {
state: {
...location.state,
course,
year
}
});
};
return (
<div className="container">
<form className="card" onSubmit={nextPage}>
```

```
<h2>Course Details</h2>
<input placeholder="Course Name" value={course} onChange={(e) => setCourse(e.target.value)} />
<input placeholder="Year" value={year} onChange={(e) => setYear(e.target.value)} />
<button>Next</button>
</form>
</div>
);
}
export default FormTwo;
```

step 3: create Summary.jsx

```
import { useLocation, useNavigate } from "react-router-dom";
function Summary() {
const { state } = useLocation();
const navigate = useNavigate();
return (
<div className="container">
<div className="card">
<h2>Summary</h2>
<p><b>Name:</b> {state?.name}</p>
<p><b>Roll:</b> {state?.roll}</p>
<p><b>Course:</b> {state?.course}</p>
<p><b>Year:</b> {state?.year}</p>
<button onClick={() => navigate("/")}>Start Again</button>
</div>
</div>
);
}
```

```
export default Summary;
```

Dynamic Routing (URL Params)

```
<Route path="/user/:id" element={<User />} />
import { useParams } from "react-router-dom";

function User() {
  const { id } = useParams();
  return <h2>User ID: {id}</h2>;
}
```

□ Important Components & Hooks

Feature	Use
BrowserRouter	Wraps the app
Routes	Holds all routes
Route	Defines path & component
Link	Navigation

Feature	Use
useNavigate	Programmatic navigation
useParams	Read URL parameters